

General Sir John Kotelawala Defence University Faculty of Computing

AI Based Career Path Prediction and Personalized Career Guidance System For Undergraduate Students

FINAL REPORT

Group Number : 45

Registration No	Name
D/BIT/24/0088	WKAMP Premachandra
D/BIS/24/0034	PHGM Dilshani
D/BCS/25/0022	KHC Vihansa
D/COE/25/0024	TDT Gamage

Contents

Abstract....4
1. Introduction....5
2. Problem Background....6
    2.1 What the problem is and who is affected....6
    2.2 Why it is important....6
    2.3 How it is currently handled and the limitations....6
    2.4 How AI can improve the situation....6
3. Problem Statement....7
4. Objectives....8
5. Selected AI Techniques....9
    5.1 Rule based reasoning (Knowledge based system)....9
    5.2 Fuzzy logic (Mamdani inference)....9
    5.3 Supervised Machine Learning (classification)....9
    5.4 Hybrid Score Fusion....10
    5.5 Explainable AI (SHAP)....10
    5.6 Skill gap analysis, similarity-based recommendation, and Generative AI....10
6. Justification of AI Techniques....11
7. System Design....13
    7.1 Architecture and Workflow....13
    7.2 Inputs....14
    7.3 Outputs....14
    7.4 Knowledge representation and data....15
    7.5 User interface design....15
8. Implementation Details....19
    8.1 Technology Stack....19
    8.2 Code Structure....20
    8.3 Fuzzy Engine....20
    8.4 Machine learning engine and Explanation....20
    8.5 Guidance generation with verified Generative AI....20
    8.6 Guidance generation with verified Generative AI....20
    8.7 Progress tracking and storage....21
    8.8 Testing infrastructure and deployment....21
9. Testing and Results....22
    9.1 Machine learning model evaluation....22
    9.2 Sample inputs and outputs....22
    9.3 Explanation and narration quality....24
    9.4 Engine behavior and reliability....24

9.5 Performance..... 24
10. Limitations and Discussion..... 25
11. Ethical and Social Considerations..... 26
12. Conclusion..... 27
References..... 28
Individual Contributions..... 30

Abstract

Undergraduate students often choose a career direction on the basis of trends, peer influence or generic advice rather than an understanding of how their own academic record, skills and interests match the requirements of different roles. University career guidance is valuable but limited in time and rarely personalized or continuous. This project presents PathFinder AI, a web-based decision-support prototype that ranks nine technology career paths for an undergraduate student, explains why each career was recommended, identifies prioritized skill gaps, recommends certifications that cover those gaps, and produces a sequenced learning pathway.

The system is a hybrid of three AI reasoning engines whose scores are fused into one ranking: a transparent rule-based engine (26 IF–THEN rules stored in a JSON knowledge base), a Mamdani fuzzy inference system (scikit-fuzzy, 10 rules over four linguistic input variables) and a supervised Random Forest classifier (scikit-learn), with a Decision Tree as a baseline. Final scores combine the engines as 30% rule, 30% fuzzy and 40% machine-learning probability. Recommendations are explained with SHAP TreeExplainer attributions and a rule-firing trace. Skill gaps are ranked by their size relative to each skill's scale and matched to a 19-item certification catalogue with gap-weighted cosine similarity. An optional Generative AI layer (Google Gemini) narrates the results in plain language, but its output is automatically validated against the computed facts and replaced by template text if it introduces any new number, certification or provider.

On a synthetic benchmark cohort of 405 student profiles across nine careers, the Random Forest reached 95.1% test accuracy (94.8% ± 1.6% five-fold cross-validation) against 80.2% for the Decision Tree baseline. Because the cohort is generated from the same career benchmarks as the rule and fuzzy engines, these figures demonstrate internal consistency rather than real-world accuracy. Four representative sample students were each ranked to their intended career, a full assessment without Gemini completes in under about one second, and an automated test suite (11 pipeline modules and a five-step headless UI test) runs on every change. The report closes with the main limitations, notably synthetic training data and a small certification catalogue, and with the ethical safeguards built into the design.

1. Introduction

Choosing a first career direction is one of the most consequential decisions an undergraduate makes, yet it is usually made with incomplete information. Students can see their own grades and skills, and they can read job descriptions, but they rarely get a structured comparison between the two, an explanation of where they stand, or a concrete plan for what to learn next.

Artificial Intelligence is well suited to this kind of decision support. Knowledge about what each career requires can be encoded explicitly as rules; the gradual, imprecise nature of skill levels can be captured with fuzzy logic; and patterns in student profiles can be learned with supervised machine learning. Combining these approaches, and explaining the result, lets a system give recommendations that are both personalized and understandable.

This report describes PathFinder AI, the prototype developed by Group 45 for the Essentials of Artificial Intelligence module. Sections 2 to 4 present the problem background, the problem statement and the objectives. Sections 5 and 6 describe the AI techniques and justify their selection. Section 7 presents the system design and Section 8 the implementation. Section 9 reports testing and results, Section 10 discusses limitations, Section 11 covers ethical and social considerations, and Section 12 concludes. References and the individual contribution table follow.

2. Problem Background

2.1 What the problem is and who is affected

Many undergraduates are uncertain which career path fits their abilities. They may follow popular technology trends, peers or social media rather than an understanding of how their own profile connects to the requirements of different roles. The primary group affected is undergraduate students, particularly in computing related programs where career options are numerous and change quickly. Career counsellors and academic advisors are affected as well, because they must support many students with limited time.

2.2 Why it is important

A poorly informed choice can lead to time spent on the wrong electives, certifications and projects, weaker employability, and lower motivation. Conversely, knowing early which skills matter most for a target role lets a student plan electives, projects and certifications deliberately across the degree.

2.3 How it is currently handled and the limitations

- Counselling sessions and workshops provide valuable human guidance but are time limited and cannot continuously monitor how each student's profile develops.
- General career information (job descriptions, generic skill lists) is not personalized to a student's grades, skills, interests, projects and certifications.
- Self-directed online research gives access to a very large number of courses and certifications, but students find it hard to decide which skills to improve first and which resources are relevant to their goal.
- Most existing tools do not explain their recommendations, so students cannot see which factors led to a suggestion or judge whether it is trustworthy.

2.4 How AI can improve the situation

An AI-based system can analyze a complete student profile in seconds, compare it against explicit knowledge about several careers, rank multiple suitable directions rather than a single answer, explain the factors behind each recommendation, identify and prioritize skill gaps, and suggest relevant certifications and learning activities. Because the analysis is fast and repeatable, it can be re-run whenever the student improves a skill, completes a certification or changes their interests, which supports continuous rather than one off guidance. Published work on machine-learning career prediction, fuzzy logic career recommendation and explainable recommendation supports this direction [4]-[12].

3. Problem Statement

Undergraduate students lack access to personalized, data-driven career support that considers their individual academic performance, technical skills, soft skills, interests, projects and certifications. Existing guidance is often generic, depends on limited counselling time, does not explain why a career fits, and does not tell students which skills or certifications to pursue next.

This project addresses the question: how can an AI-powered system analyze an undergraduate student's academic performance, skills, interests and experience to rank suitable career paths, explain the reasons behind the recommendations, identify relevant skill gaps, and provide personalized guidance for developing the skills those careers require?

The scope is career guidance and skill development for undergraduate students in computing-related programs, across nine technology careers. The system supports, and does not replace, the student's own decision and the role of human career counsellors.

4. Objectives

The objectives below are those set in the project proposal. The status column records the outcome at final submission.

Objective	Value
Collect and analyze the student's GPA, module grades, technical and soft skills, interests, projects, experience and career preferences.	Achieved: a structured profile form with GPA, academic year, six core modules, electives, eight technical proficiencies, 24 soft skills in four pillars, work style, desired domains, internship, projects and held certifications.
Build a hybrid model combining a transparent scoring baseline with supervised machine learning to rank career paths.	Achieved: rule-based, fuzzy and Random Forest engines fused 30/30/40 into one ranking of nine careers, with a Decision Tree baseline for comparison.
Produce explainable recommendations using SHAP.	Achieved: SHAP TreeExplainer attributions for the active model, plus a rule-firing trace and soft-skill match breakdown.
Analyze skill gaps against the recommended career and prioritize them.	Achieved: gaps ranked by size relative to each skill's scale and labelled High, Medium or Low urgency.
Use Generative AI to turn gaps into a personalized, sequenced learning pathway.	Achieved: a deterministic pathway of courses, certifications, projects and activities, narrated by Gemini with automatic fact validation and a template fallback.
Adapt recommendations as the student progresses.	Achieved: an optional pseudonymous student ID stores each assessment; later runs show match changes, closed gaps and a trajectory chart; certifications already held are excluded from recommendations.
Evaluate prediction accuracy, explanation quality and usability with sample or synthetic records.	Partly achieved: model accuracy, engine behaviour, sample-student outputs, narration validation and response time were evaluated (Section 9).

5. Selected AI Techniques

The brief requires at least three AI-related concepts. PathFinder AI applies six, which work together as one pipeline (Figure 1, Section 7).

5.1 Rule based reasoning (Knowledge based system)

Expert knowledge about each career is written as IF–THEN rules in a JSON knowledge base (data/rules.json) and evaluated by a small custom inference engine. There are 26 rules: 5 common rules (three GPA bands such as R-GPA-HIGH: IF GPA ≥ 3.5 THEN +15 points, plus internship and project-portfolio rules), 19 career-specific rules (for example, R-SE-CORE: IF DSA ≥ 75 AND OOP ≥ 75 THEN +25 points for Software Engineer), a soft-skill rule that adds points in proportion to the share of the career's required soft skills the student has, and a work-style rule (+5 when the preferred work style suits the career). The engine returns a score between 0 and 1 for each career together with the list of rules that are fired, which is shown to the student as a transparent reasoning trace.

5.2 Fuzzy logic (Mamdani inference)

Skill levels are gradual rather than crisp, so a Mamdani fuzzy inference system built with scikit-fuzzy estimates how suitable each career is. It has four input variables with overlapping membership functions: GPA (low, medium, high), technical mastery relative to the career's benchmark (weak, moderate, strong), soft-skill match (low, moderate, high) and work-style fit (mismatch, match). Ten rules (F1–F10) map these to a suitability output (poor, fair, good, excellent), for example: IF tech_mastery is strong AND gpa is high AND soft_match is high THEN suitability is excellent. The output is defuzzified with the centroid method. Every technical-mastery term has at least one rule that always fires, so the output is always defined.

5.3 Supervised Machine Learning (classification)

A Random Forest classifier (80 trees, maximum depth 10) learns the relationship between a 20 feature student vector (GPA, year, six core modules, eight technical skills, internship, projects, work style and soft skill affinity) and the nine career classes and outputs a probability for each career. A single decision tree is provided as a baseline, and the prototype's Model Studio lets the user retrain either model on the benchmark dataset or on an uploaded CSV/Excel cohort and compare them with five-fold cross-validation.

5.4 Hybrid Score Fusion

The three engine scores are combined for every career as: final score = 0.30 × rule score + 0.30 × fuzzy score + 0.40 × ML probability. Careers are ranked by this score and given a confidence band: High (≥ 0.63), Moderate (≥ 0.46) or Exploring. The thresholds were calibrated on 600 synthetic profiles so that roughly the top fifth of top-ranked results are labelled High.

5.5 Explainable AI (SHAP)

SHAP TreeExplainer [2] computes, for the individual student, how much each feature pushed the active model's probability for the recommended career up or down. The contributions sum exactly to the model's prediction minus its average, so the chart is a faithful explanation of the machine-learning component. Together with the rule trace and the soft-skill breakdown, this explains all three engines.

5.6 Skill gap analysis, similarity-based recommendation, and Generative AI

A skill gap layer compares the student's skills with the recommended career's benchmark and ranks the gaps by their size as a share of each skill's own scale (module marks out of 100, proficiencies out of 5). A content-based recommender then scores each of 19 certifications with gap-weighted cosine similarity between the gap vector and the certification's skill vector, adjusted for domain relevance and the student's academic year; certifications in the career's own domains always rank first and certifications the student already holds are excluded. Finally, a deterministic planner orders gaps, certifications, projects and activities into a phased learning pathway, and Google Gemini (a large language model) rewrites the summary and pathway in natural language.

6. Justification of AI Techniques

Table 2. Justification of the selected techniques

Technique	Why it suits this problem	How it contributes to the solution
Rule-based reasoning	Career requirements are well understood by people and can be written down explicitly; rules work without any training data and are fully transparent.	Gives an interpretable baseline score and a human-readable trace of exactly which conditions were met.
Fuzzy logic	Skill and performance levels are gradual (a GPA of 3.4 is neither clearly medium nor clearly high); crisp thresholds would make small differences change results abruptly.	Produces smooth suitability scores that reflect partial membership, making the assessment more realistic and stable.
Random Forest / Decision Tree	Interactions between many features are hard to capture fully with hand-written rules; ensembles of trees handle mixed numeric features, are robust to noise and work well on small tabular datasets.	Learns patterns from labelled profiles and contributes a calibrated probability per career; the Decision Tree baseline shows the benefit of the ensemble.
Score fusion	Each engine has different strengths and weaknesses; combining them reduces reliance on any single method.	Produces one ranked list of careers with a confidence band.
SHAP	Recommendations influence real academic and career decisions, so students need to know why. SHAP gives consistent per student attributions for tree models.	Shows which features drove the top recommendation, building trust and helping students act on the result.
Gap analysis and cosine similarity	Guidance is only useful if it says what to do next; cosine similarity is a simple, explainable way to match a gap profile to skill-tagged resources.	Turns the ranking into prioritized skill gaps and certification recommendations with a coverage percentage.
Generative AI (Gemini)	Students respond better to clear, natural explanations than to raw scores; an LLM is good at rephrasing structured facts.	Narrates the summary and learning pathway. It never changes the ranking, and its output is validated and replaced by template text if it adds unsupported facts.

Alternatives were considered. A neural network was not used because the dataset is small and tabular and because tree models give more direct explanations. A purely rule-based system would be transparent but brittle, and a purely machine-learning system would depend entirely on training data that, in this project, is synthetic. The hybrid design balances transparency, flexibility and learned behavior.

7. System Design

7.1 Architecture and Workflow

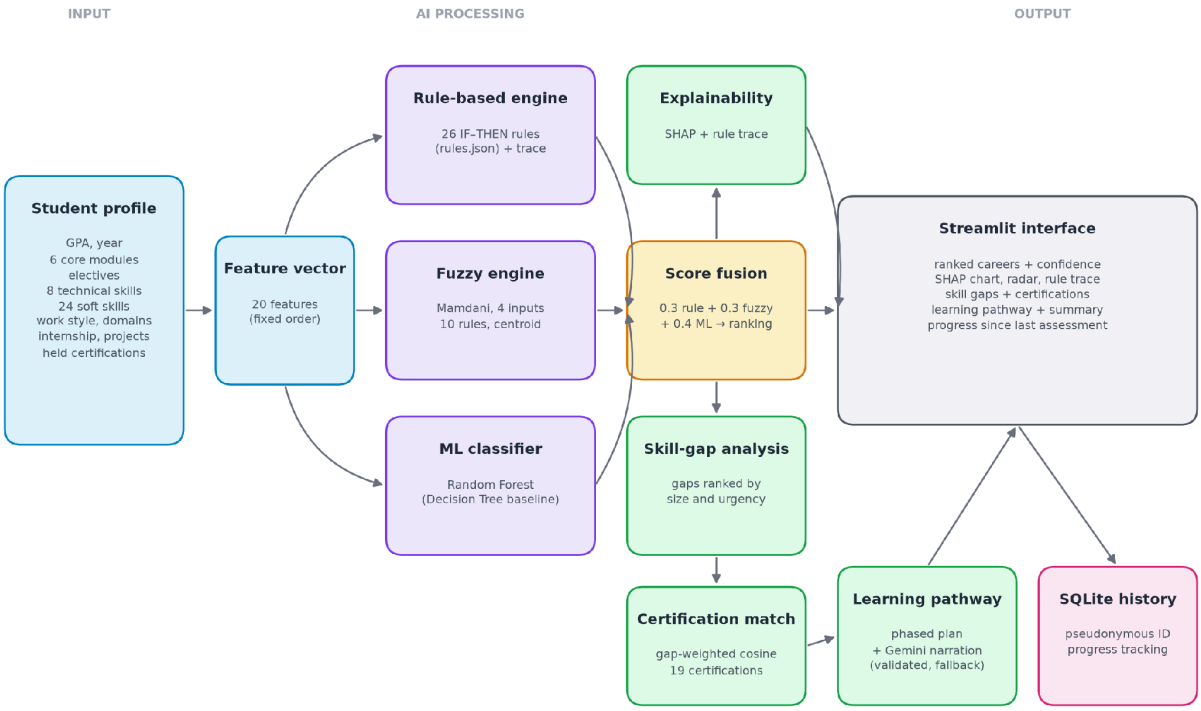


Figure 1. PathFinder AI architecture: inputs, AI processing and outputs

The workflow follows an input-process-output structure. The student completes the profile form; the profile is converted into a fixed-order feature vector; the three engines score all nine careers independently; the scores are fused into a ranked list; the top career is explained, its skill gaps are analyzed, certifications are matched, and a learning pathway is generated and narrated. The assessment is optionally stored under a pseudonymous ID so that later assessments can be compared.

7.2 Inputs

Table 3. System inputs

Input group	Details
Academic	Cumulative GPA (0.00–4.00), academic year, marks (0–100) for six core modules: Data Structures & Algorithms, OOP, DBMS, OS & Networks, Software Engineering Principles, Mathematics & Statistics; optional elective modules with marks.
Technical skills	Self-rated proficiency (1 Novice – 5 Expert) in Python, Java/C++, SQL, web stack, cloud/Docker, ML/AI frameworks, mobile development and cybersecurity.
Soft skills	24 work strengths in four pillars: People & Leadership, Ideas & Innovation, Data & Systems, Execution & Practical.
Preferences and experience	Work style (Technical Specialist, Consulting/Management, R&D/Creative), desired industry domains, internship completed, number of technical projects, certifications already held (free text).
Tracking	Optional pseudonymous student ID used to compare assessments over time.

7.3 Outputs

- Ranked list of nine careers with match percentage and confidence band; the top three are shown as cards with their archetype and role description.
- Explanations: SHAP feature contributions, a competency radar against the career benchmark, the rule firing trace and the soft-skill match breakdown.
- Prioritized skill gaps with current level, target level and urgency.
- Three recommended certifications with gap-coverage percentage and links to the official exam page and preparation course.
- A phased learning pathway and an executive summary (Gemini-narrated and fact-checked, or template text).
- Progress since the previous assessment (when a student ID is used) and a history log with CSV export.

7.4 Knowledge representation and data

Table 4. The nine careers and five archetypes

Career	Archetype
Software Engineer	Engineering & Architecture
AI Engineer	Engineering & Architecture
CAD-CAM Engineer	Engineering & Architecture
Data Scientist	Data & Analytics
Cloud Architect	Infrastructure, Systems & Security
Cybersecurity Specialist	Infrastructure, Systems & Security
UX Designer	Creative, UI/UX & Media
Game Developer	Creative, UI/UX & Media
IT Business Analyst	Management & Consulting

Each career in the knowledge base has benchmark skill levels, required soft skills, preferred work styles, a description and a role dynamic. The rule base is stored as JSON so that rules can be read, reviewed and changed without modifying code. The certification catalogue lists 19 certifications, each tagged with domains, level, target years and a weighted skill vector.

Training data is a synthetic benchmark cohort generated around each career's benchmark skills (45 profiles per career, 405 in total, fixed random seed), split 80/20 with stratification (324 training, 81 test rows). The 20 features are listed in Section 5.3. Assessments are stored in a local SQLite database.

7.5 User interface design

The interface is a Streamlit web application with four areas: Profile Intake (the form, a live profile snapshot and quick-load sample profiles), Guidance & Roadmap (the results), Dataset & Model Studio (dataset inspection, upload, training and model comparison) and History Log (stored assessments, per-student trajectory and CSV export). A full-screen loading screen shows which step is running during an assessment, and the results page opens automatically when it finishes.

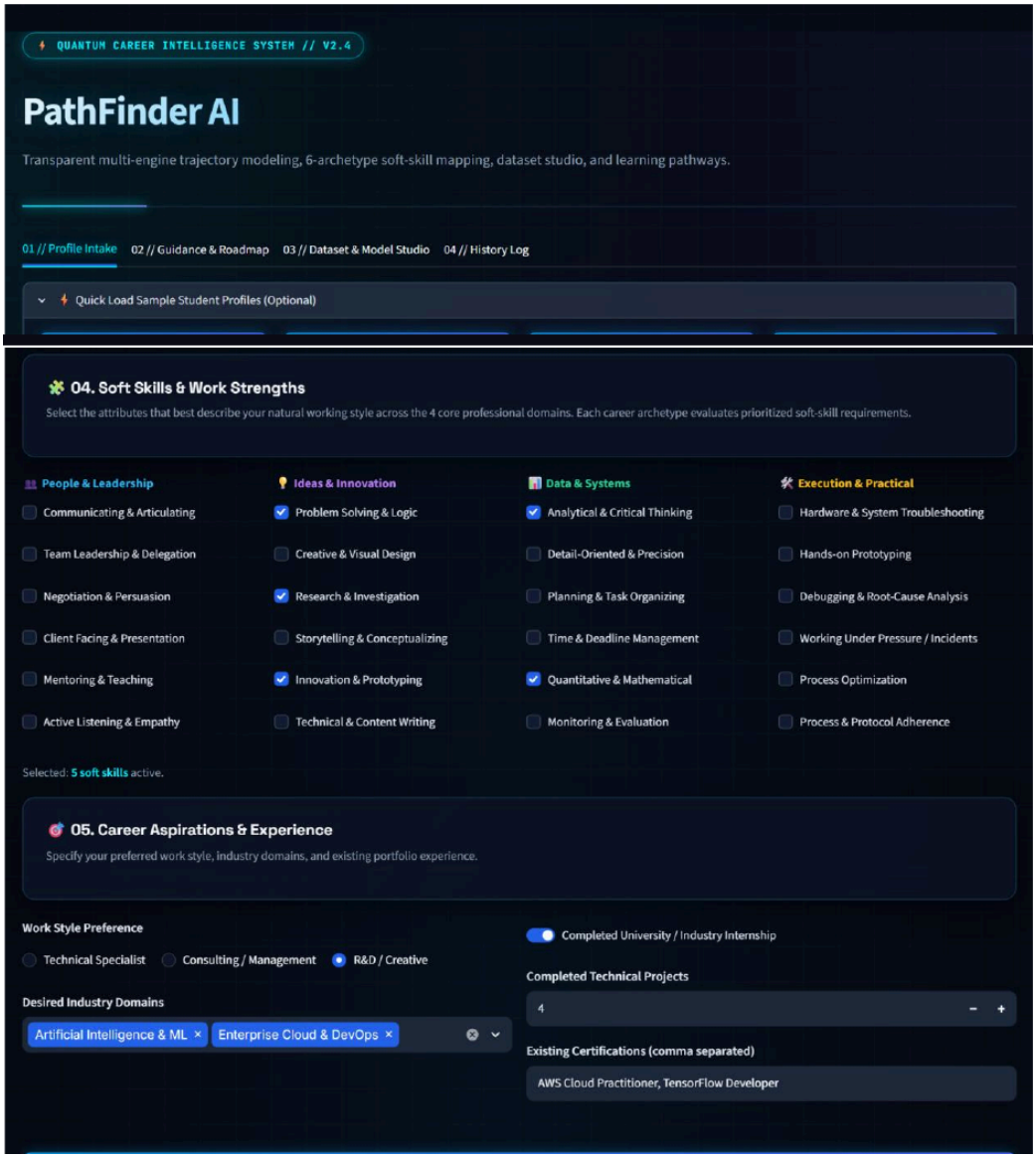


Figure 2. Profile Intake tab: quick load sample profiles, soft-skill selection, career preferences, and the inference button

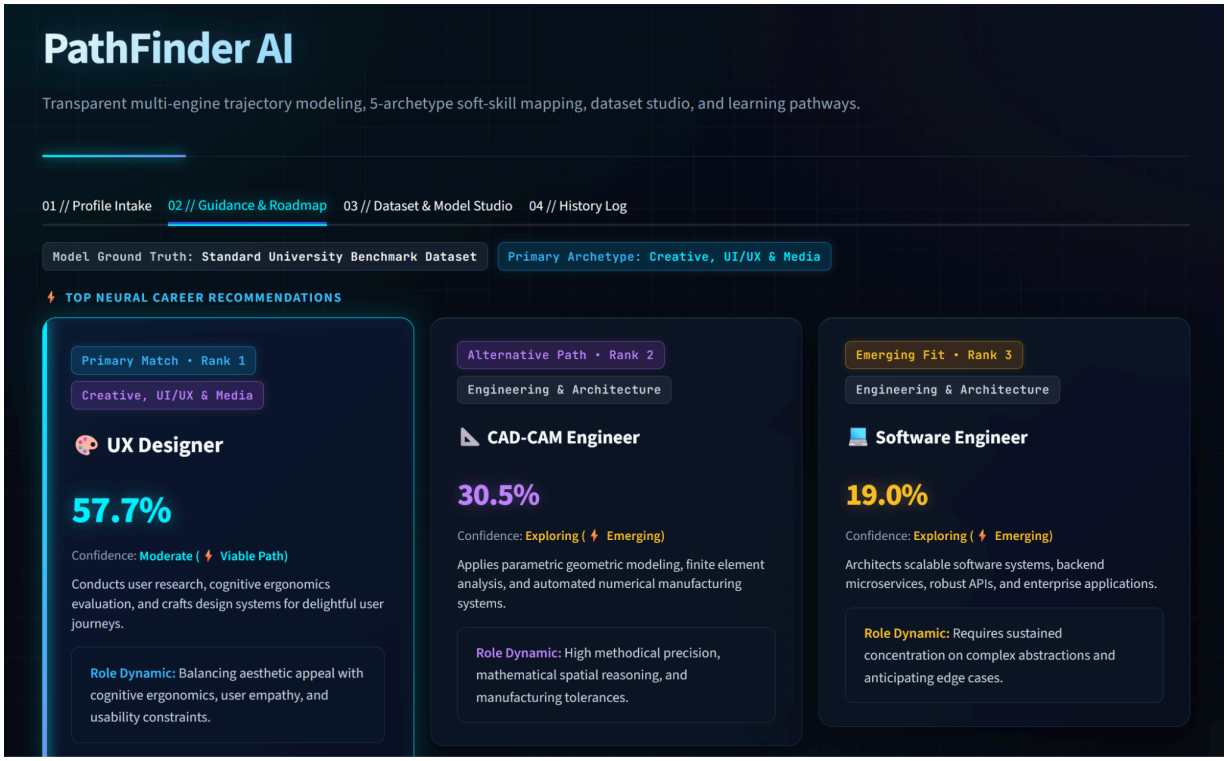


Figure 3. An overview of the top career recommendations



Figure 4. The AI advisory summary, breakdown of career alignment and result explanation

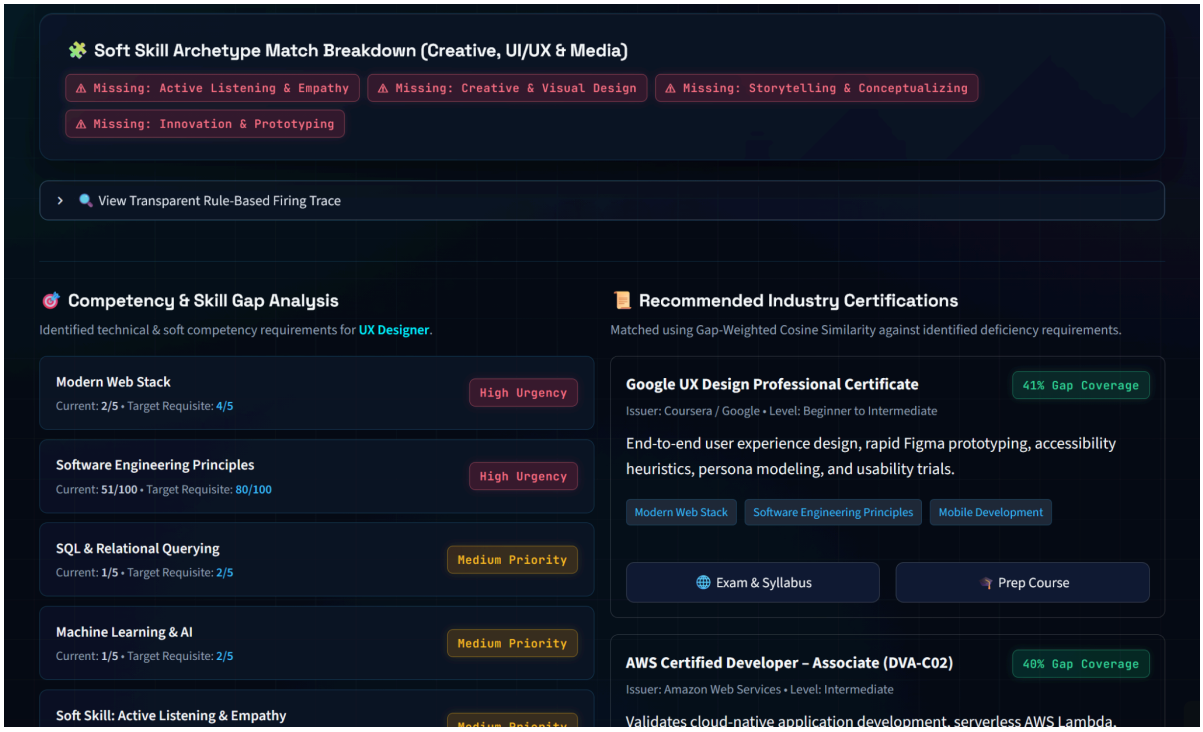


Figure 5. The soft skill match breakdown, competency & skill gap analysis, and certification recommendations

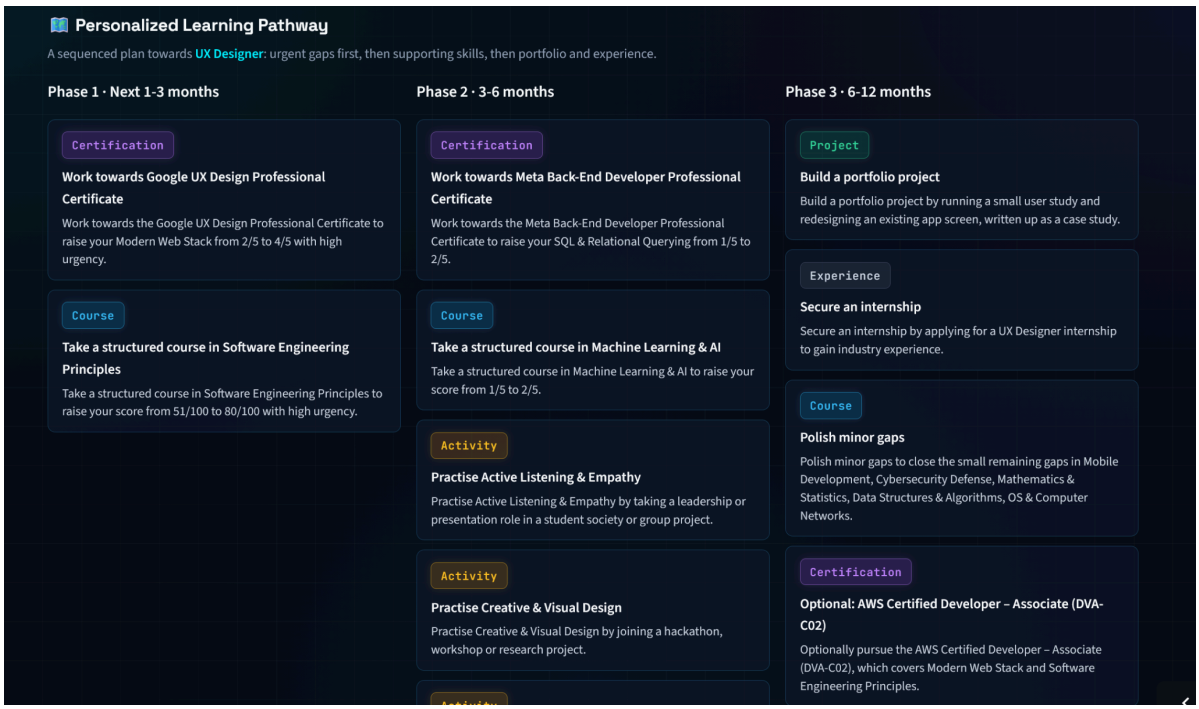


Figure 6. The personalised learning pathway

8. Implementation Details

8.1 Technology Stack

Table 5. Technology stack

Layer	Technology
Language and environment	Python 3.11 (tested on 3.11–3.14), dependencies pinned in requirements.txt
User interface and charts	Streamlit 1.65.0, Plotly 7.1.0
Data handling	pandas 3.0.6, NumPy 2.4.6
Rule-based reasoning	Custom Python evaluator over data/rules.json
Fuzzy logic	scikit-fuzzy 0.5.0 (with networkx 3.6.1)
Machine learning	scikit-learn 1.9.1 (RandomForestClassifier, DecisionTreeClassifier), joblib 1.6.0
Explainability	SHAP 0.51.0 (TreeExplainer)
Generative AI	Google Gemini API through google-genai 2.28.0 (model gemini-3.5-flash-lite), with template fallback
Storage	SQLite; JSON and CSV data files
Quality and delivery	GitHub, GitHub Actions continuous integration, Streamlit Community Cloud hosting

8.2 Code Structure

The Streamlit entry point (app.py) contains only the user interface. The AI logic lives in a src package with one module per responsibility: profile (student profile and feature vector), knowledge_base (careers and soft skills), rule_engine, fuzzy_engine, ml_engine (dataset generation, training, comparison), fusion, explainability (SHAP), skill_gaps, recommender, guidance (pathway, template text and Gemini narration), progress and database (tracking and SQLite), and train (exports the trained models and metrics to the models folder). Data files are in data and tests in tests.

8.3 Fuzzy Engine

Each rule in rules.json has an identifier, a condition (a single comparison or an all/any combination on profile fields, skills or electives), the points awarded and an explanation text. Rules in the same exclusive group (the GPA bands) behave as an IF / ELSE IF chain, so only the first matching band awards points. The engine adds the points of the common and career rules, the soft-skill rule and the work-style rule, divides by the maximum of 100 and caps the result at 1 to give a 0–1 score, and records the text of every rule that fired.

8.4 Machine learning engine and Explanation

The fuzzy system is built once with scikit-fuzzy's control API from the membership functions and ten rules described in Section 5.2, and evaluated for each career with the student's GPA, technical mastery (the student's skills relative to the career benchmark), soft-skill match and work-style fit. The defuzzified centroid is the fuzzy suitability score.

8.5 Guidance generation with verified Generative AI

The model is trained in memory on the benchmark cohort when the app starts (under one second), which avoids version problems when loading saved model files on the hosting platform. A separate script exports identical models to the models folder together with a metrics file, and a test confirms that the exported Random Forest matches the one the app trains. SHAP TreeExplainer is applied to whichever model is active, and the top seven contributions are shown.

8.6 Guidance generation with verified Generative AI

The learning pathway is built deterministically: urgent gaps first, each paired with a recommended certification if the certification strongly covers that skill (weight ≥ 0.8) or otherwise with a course, then supporting skills, then portfolio projects and experience. Only these computed facts are sent to Gemini, which returns a summary and one sentence per pathway step as structured JSON. The output is accepted only if it keeps every step in order, introduces no number that is not in the facts,

and mentions no certification or provider that the engines did not recommend. Any failure (no API key, network error, timeout or failed validation) falls back to template text, so Gemini can never change the recommendation itself. Gemini responses are cached by their input facts.

8.7 Progress tracking and storage

If the student enters an ID, it is normalized (case- and space-insensitive) and each assessment is saved with the career scores, profile and gaps. The next assessment under the same ID is compared with the previous one: change in match for the top career, gaps closed, profile changes and a chart of match percentages across assessments. Certifications on the student lists as already held are recognized and not recommended again.

8.8 Testing infrastructure and deployment

An automated pipeline test checks all engines across 11 modules, and a headless UI smoke test (Streamlit AppTest) drives the real application: first load, two assessments with a student ID, model training and comparison, and the tab navigation. Both run automatically in GitHub Actions on every change, with a pyflakes code check, and the protected main branch requires them to pass. The prototype is deployed on Streamlit Community Cloud.

9. Testing and Results

9.1 Machine learning model evaluation

Table 6. Classifier accuracy on the synthetic benchmark cohort (405 profiles, 9 classes, stratified 80/20 split)

Model	Train accuracy	Test accuracy	5-fold CV accuracy
Random Forest (80 trees, depth 10)	100.0%	95.1%	94.8% ± 1.6%
Decision Tree baseline (depth 10)	99.4%	80.2%	82.5% ± 2.5%

The Random Forest outperforms the single Decision Tree by about 15 percentage points on the held-out test set and generalizes far better (the Decision Tree's near-perfect training accuracy shows overfitting). Per-career F1-scores for the Random Forest range from 0.89 (Cloud Architect, Cybersecurity Specialist) to 1.00 (Software Engineer, UX Designer, IT Business Analyst); the four misclassifications are all between related careers (Cloud Architect and Cybersecurity Specialist in both directions, AI Engineer to Data Scientist, Game Developer to CAD-CAM Engineer). The most important Random Forest features were DSA, Mathematics & Statistics, OS & Networks, OOP and DBMS.

Important caveat: the cohort is synthetic and generated from the same career benchmarks used by the rule and fuzzy engines, so these figures show that the model learns the intended structure consistently. They are not evidence of accuracy on real students.

9.2 Sample inputs and outputs

Four representative student profiles (the quick-load samples in the prototype) were run through the complete pipeline. Table 7 summarizes their inputs and Table 8 the generated outputs.

Table 7. Sample inputs

Sample student	Academic and style	Core modules (/100)	Selected skills (/5)
AI Engineer	GPA 3.82, 3rd Year, R&D / Creative	DSA 92, OOP 88, DBMS 82, OS/Net 78, SE 85, Math 94	Python 5, SQL 4, Web 3, Cloud 4, ML 5, Security 2
Cloud Architect	GPA 3.55, 4th Year, Technical Specialist	DSA 80, OOP 82, DBMS 88, OS/Net 94, SE 86, Math 72	Python 4, SQL 4, Web 3, Cloud 5, ML 2, Security 4

Sample student	Academic and style	Core modules (/100)	Selected skills (/5)
Full-Stack Engineer	GPA 3.60, 3rd Year, Technical Specialist	DSA 88, OOP 90, DBMS 85, OS/Net 78, SE 92, Math 78	Python 4, SQL 4, Web 5, Cloud 3, ML 2, Security 2
IT Business Analyst	GPA 3.65, 3rd Year, Consulting / Management	DSA 70, OOP 72, DBMS 86, OS/Net 68, SE 94, Math 80	Python 3, SQL 5, Web 2, Cloud 2, ML 2, Security 2

Table 8. Generated outputs

Sample student	Top 3 careers (match %)	Top SHAP drivers (pts)	Main skill gaps
AI Engineer	AI Engineer 87.9%; Data Scientist 54.8%; Software Engineer 53.5% (top: High)	ML/AI +27.5, DSA +8.2, Python +7.6	Java / C++ Systems (3/5 → 3.5/5); Cybersecurity Defense (2/5 → 2.5/5)
Cloud Architect	Cloud Architect 78.2%; Cybersecurity Specialist 60.9%; Software Engineer 52.4% (top: High)	Cloud/Docker +16.7, OS & Networks +12.9, Cybersecurity +6.5	None (benchmark met)
Full-Stack Engineer	Software Engineer 87.8%; CAD-CAM Engineer 49.6%; IT Business Analyst 46.5% (top: High)	OOP +9.7, Web Stack +9.4, DSA +7.9	Innovation & Prototyping (soft skill, missing); Cybersecurity Defense (2/5 → 2.5/5)
IT Business Analyst	IT Business Analyst 70.4%; Data Scientist 47.2%; UX Designer 45.0% (top: High)	Java/C++ +5.2, SE Principles +4.9, DBMS +4.5	Mobile Development (1/5 → 2/5); Analytical & Critical Thinking (soft skill, missing)

All four samples were ranked to their intended career with a High confidence band (match 70.4%–87.9%), and the runners-up are plausible neighboring careers (for example Data Scientist and Software Engineer for the AI Engineer sample, Cybersecurity Specialist for the Cloud Architect sample). The SHAP drivers match intuition: ML/AI proficiency for the AI Engineer, Cloud/Docker and OS & Networks for the Cloud Architect, OOP and web stack for the Software Engineer. The rule traces show the expected career rules firing, such as R-AI-MATH, R-CLOUD-NET, R-SE-CORE and R-BA-PM. The Cloud Architect sample already met every benchmark, and the system correctly reported no gaps.

Two weaknesses were also visible. First, the rule engine gave the maximum score of 1.0 to the top two or three careers in several samples, so separation between careers comes mainly from the fuzzy and machine learning engines. Second, when a strong profile has only minor gaps, the small catalogue means some recommended certifications come from outside the target career's domain (for example Unity and Oracle Java certifications for the AI Engineer sample). Both are discussed in Section 10.

9.3 Explanation and narration quality

SHAP explanations were checked automatically: for every tested profile, the base value plus the feature contributions equals the model's predicted probability. For the Gemini narration, a trial of 12 real API calls produced output that passed every validation check in all 12 cases, at about 3 seconds per call. When the Gemini service was overloaded during testing (HTTP 503 and 504 responses), the system fell back to template text as designed and the assessment still completed.

9.4 Engine behavior and reliability

- **Rule base:** after moving the rules from code into rules.json, 27,000 rule evaluations were compared with the previous implementation and were identical.
- **Fuzzy engine:** a defect found at the progress review (a perfect GPA or soft-skill match scoring zero because of a membership-function edge) was fixed, and a test now checks that a perfect GPA and soft-skill match never scores lower.
- **Score fusion:** an earlier ×2.2 stretch of the ML probability saturated one career at 1.0 for 59% of students; fusion now uses raw probabilities, and the confidence bands were recalibrated on 600 profiles.
- **Refactoring safety:** moving the engines into the src package was verified as behaviour-identical on 300 random profiles (scores, rule traces, SHAP values, gaps, certifications, narrative and pathway).
- **Automated tests:** the 11 pipeline test modules and the 5-step UI smoke test pass on Python 3.11, 3.12, 3.13 and 3.14, and run on every change in GitHub Actions.

9.5 Performance

A full assessment (inference, SHAP, gaps, certifications and pathway) took 0.03–0.07 seconds per sample student after the first call, and about one second for the first call, which includes training the model. With Gemini enabled, an assessment normally takes about 3 seconds longer; Gemini results are cached, and requests time out after 20 seconds with a template fallback.

10. Limitations and Discussion

- **Synthetic training data.** The classifier is trained and evaluated on synthetic profiles derived from the same benchmarks as the rule and fuzzy engines. High accuracy therefore shows internal consistency, not real-world validity. Real, consented student outcome data would be needed to validate the recommendations.
- **Self-reported inputs.** Technical proficiency, soft skills and project counts are entered by the student and are not verified, so results are only as accurate as the self-assessment.
- **Knowledge-base coverage.** Only nine careers and 19 certifications are modelled. The small catalogue (for example only one CAD-CAM certification) can lead to certifications from neighbouring domains being recommended when a profile has few gaps.
- **Score saturation.** The rule engine often awards the maximum score to several careers for strong students, and fuzzy scores cluster around 0.70 for moderate profiles, which reduces how well those engines separate careers. Rule weights and membership functions could be tuned with real data.
- **Explanation scope.** SHAP explains the machine-learning component (40% of the final score); the rule and fuzzy components are explained through the rule trace and the inputs rather than a single unified attribution.
- **Dependence on an external LLM**. Gemini adds latency and can be unavailable. The validation and template fallback keep the system reliable, but narration quality then drops to the template text.
- **Storage on the hosted demo.** On Streamlit Community Cloud the SQLite history is shared by all visitors and resets when the app restarts, which is acceptable for a prototype but not for real deployment.

Overall, the prototype meets the goal of demonstrating how several AI techniques can be combined into an explainable career-guidance tool. The hybrid design proved useful in practice: when one engine saturated or was uncertain, the others still separated the careers, and every number shown to the student can be traced to a rule, a fuzzy rule or a SHAP contribution. The most valuable next steps are collecting real (consented and anonymized) student data, extending the career and certification catalogues, tuning the rule and fuzzy parameters against that data, and completing a user study.

11. Ethical and Social Considerations

- Data privacy. The application collects information about academics and skills, and the use of student ID is optional, and needs to be a pseudonym, not a name. Only facts that are derived from the input information (best career, matching score, GPA, best module, matching soft skills, best gaps, suggested certifications and pathway actions) are communicated to the Gemini system, but not the student ID and the free text information. Since all the history of the hosted demo is accessible by anyone using the same ID, users are informed of that in the History page.
- Bias and fairness. The model trains synthetic data created by the group, meaning that it has incorporated the group's biases for each profession. Personal features like gender, age, or background information are not collected or considered. The results have to be reviewed for bias prior to deployment among different student groups.
- Accuracy and reliability. Each recommendation comes with a confidence interval, and the report and interface make clear that results are provided by a prototype which has been trained on synthetic data. Generative AI output is verified against the calculations so that the model can't create any figures or providers.
- Misuse. The tool must never be used to control the options of the student, determine access to course work, or screen candidates for employment. This tool is meant to educate the student rather than to brand them.
- Human responsibility. The final decision about what career a student will take will be made by them and assisted by their advisors and counselors. The information is meant to make it easy for an advisor to question a recommendation.
- Limitations of automated decisions. There are certain things that cannot be known by an algorithm such as motivation, individual situations, and the labor market. Recommendations are simply something to think about.

12. Conclusion

PathFinder AI shows that combining rule-based reasoning, fuzzy logic and supervised machine learning, and explaining the result with SHAP and a rule trace, can produce career recommendations that are personalized, transparent and actionable. Beyond the ranking, the system identifies prioritized skill gaps, recommends certifications that cover them, produces a sequenced learning pathway with fact-checked Generative AI narration, and tracks a student's progress across assessments. The prototype meets six of the seven proposal objectives in full and the seventh in part, with model accuracy of 95.1% on the synthetic benchmark (against 80.2% for a Decision Tree baseline), correct rankings for all four sample students and sub-second response times. Its main limitation is the use of synthetic data; with real, ethically collected student data and a wider knowledge base, the same architecture could become a practical support tool for students and career counsellors.

References

[1] Scikit-learn developers (2026) scikit-learn: Machine Learning in Python. Available at: https://scikit-learn.org/

[2] Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30.

[3] scikit-fuzzy developers (2026) scikit-fuzzy documentation. Available at: https://scikit-fuzzy.readthedocs.io/

[4] Trujillo, F., Pozo, M. and Suntaxi, G. (2025) 'Artificial intelligence in education: a systematic literature review of machine learning approaches in student career prediction', Journal of Technology and Science Education, 15(1), pp. 162–185.

[5] José-García, A. et al. (2023) 'C3-IoC: a career guidance system for assessing student skills using machine learning and network visualisation', International Journal of Artificial Intelligence in Education, 33, pp. 1092–1119.

[6] Razak, T.R., Hashim, M.A., Noor, N.M. et al. (2014) 'Career path recommendation system for UiTM Perlis students using fuzzy logic', 2014 5th International Conference on Intelligent and Advanced Systems (ICIAS).

[7] Zhang, L. (2024) Predicting and recommending of student career aspirations using machine learning models. University of California, eScholarship.

[8] Iorzua, J.T., Moses, T., Eke, C.I. et al. (2025) 'A machine learning based approach to course and career recommendation system: a systematic literature review', Journal of Computing Theories and Applications.

[9] Rasca, E., Villa, A., Maddalena, A. and Boccuzzi, G. (2025) 'Machine learning for post-diploma educational and career guidance: a scoping review in AI-driven decision support systems', Frontiers in Education.

[10] P, H., Malathi, K., K, K., S, G. and V, S.K. (2026) 'Career Compass: an AI-driven skill-aware career guidance framework for personalized student recommendations', 2026 9th International Conference on Inventive Computation Technologies (ICICT), pp. 1300–1305. doi:10.1109/ICICT68280.2026.11510984.

[11] Roy, M. and Sharma, K. (2026) 'AI-powered career guidance: a scalable model for personalized recommendations', IEEE Access, 14, pp. 20927–20941. doi:10.1109/ACCESS.2026.3661573.

[12] Silpa, B.L., Kiran, A., Thrimurthulu, V., Baba, M.A. and Mohammed, F. (2026) 'AI-enhanced career guidance system for personalized career pathways', 2026 International Conference on Connected Intelligence for Industrial Applications (CI2A), pp. 1–10. doi:10.1109/CI2A69097.2026.11577009.

[13] Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32.

[14] Quinlan, J.R. (1986) 'Induction of decision trees', Machine Learning, 1(1), pp. 81–106.

[15] Zadeh, L.A. (1965) 'Fuzzy sets', Information and Control, 8(3), pp. 338–353.

[16] Mamdani, E.H. and Assilian, S. (1975) 'An experiment in linguistic synthesis with a fuzzy logic controller', International Journal of Man-Machine Studies, 7(1), pp. 1–13.

[17] Lundberg, S.M. et al. (2020) 'From local explanations to global understanding with explainable AI for trees', Nature Machine Intelligence, 2(1), pp. 56–67.

[18] Pedregosa, F. et al. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825–2830.

[19] Streamlit (2026) Streamlit documentation. Available at: https://docs.streamlit.io/

[20] Google (2026) Gemini API documentation. Available at: https://ai.google.dev/gemini-api/docs

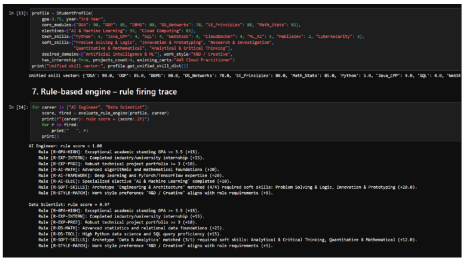
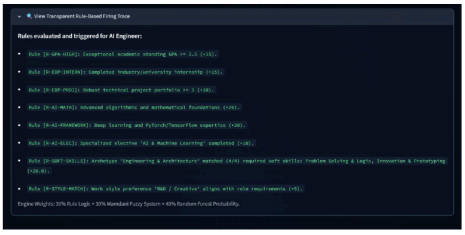
Individual Contributions

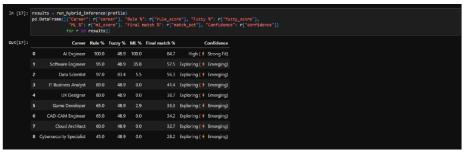
ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	WKAMP Premachandra	**Registration No.**	D/BIT/24/0088
Degree / Intake	Information Technology / 2024	**Group No.**	45
Project Title	PathFinder AI: An AI-Based Career Path Prediction and Personalised Career Guidance System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
1. Rule evaluator & firing trace	Built the inference engine that reads the rules, handles exclusive rule groups (GPA bands as IF / ELSE IF), adds career, soft-skill and work-style points, normalises the result to a 0-1 score per career, and records every fired rule as a transparent trace shown to the student.	Rule-based reasoning Python JSON rule base	 *Progress Report Fig 12*  *Progress Report Fig 23*

2. Fuzzy inference system	Designed the Mamdani fuzzy system: triangular and trapezoidal membership functions for GPA, technical mastery ratio, soft-skill match and work-style fit; ten rules producing a suitability score by centroid defuzzification; year-readiness adjustment. Produced the membership plots and suitability table for the sample profile.	Fuzzy logic (Mamdani) scikit-fuzzy Matplotlib	 *Progress Report Fig 13* *Final Report Sec 5.2, 8.4*
3. Hybrid score fusion	Implemented the fusion of rule, fuzzy and Random Forest scores (30% / 30% / 40%) into one ranked list of nine careers with High, Moderate and Exploring confidence bands. After testing, removed the x2.2 probability scaling that saturated scores and recalibrated the bands on 600 profiles.	Hybrid score fusion Python, pandas	 *Progress Report Fig 14* *Final Report Sec 5.4, 9.4*
4. Engine testing & bug fixing	Identified and reproduced the fuzzy boundary bug (perfect GPA or soft-skill match scoring zero) and the ML probability saturation, fixed both, compared 27,000 rule evaluations with the previous implementation and added a regression test that a perfect match never scores lower.	Python Automated tests (test_pipeline.py) GitHub Actions	*Progress Report Fig 28 and Sec 8* *Final Report Sec 9.4*
5. Environment setup, integration & deployment	Set up the Python environment with pinned dependencies (requirements.txt), integrated the rule, fuzzy, machine-learning, explainability, gap and recommender modules into the application, and configured Streamlit Community Cloud deployment so every member and evaluator can open the live prototype.	Python virtual environment Git / GitHub Streamlit Community Cloud	*Live prototype:* *https://eai-career-guidance-ai.streamlit.app/* *Source code:* *https://github.com/muditha Premachandra/career-guidance-ai.git* *Final Report Sec 8.1, 8.8*

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 17/10/2026

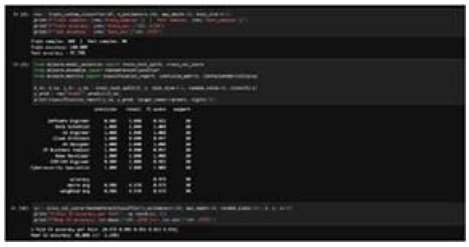
Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	TDT Gamage	**Registration No.**	D/COE/25/0024
Degree / Intake	Computer Engineering / 2025	**Group No.**	45
Project Title	PathFinder AI: An AI-Based Career Path Prediction and Personalised Career Guidance System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
1. Random Forest career classifier	Trained the multi-class Random Forest (80 trees in the app, maximum depth 10) on the 20-feature vector with a stratified 80/20 split, producing a probability for each of the nine careers that feeds the score fusion.	Supervised machine learning scikit-learn RandomForestClassifier pandas	 *Progress Report Fig 8* *Final Report Sec 5.3*
2. Model evaluation	Evaluated the classifier with accuracy, per-class precision / recall / F1, five-fold cross-validation and a confusion matrix; analysed misclassifications between related careers and the feature importances.	scikit-learn metrics Cross-validation Matplotlib	 *Progress Report Fig 9* *Final Report Sec 9.1, Table 6*

3. Retraining & feature importance in Model Studio	Implemented retraining on the benchmark or an uploaded CSV/Excel cohort with adjustable trees, depth and test split, reporting train/test accuracy and global feature importances; wrote the script that exports models and metrics and the test that the exported model matches the app's model.	scikit-learn joblib Plotly	 *Progress Report Fig 11* *Final Report Sec 8.5*
4. Decision Tree baseline	Added a Decision Tree baseline and compared it with the Random Forest on test and cross-validation accuracy: Random Forest 95.1% test (94.8% CV) against 80.2% (82.5% CV), showing the benefit of the ensemble.	scikit-learn DecisionTreeClassifier Model comparison	*Final Report Sec 9.1, Table 6*
5. SHAP explainability	Built the feature-contributio n explanation for the top career, first as a SHAP-style proxy and then with real SHAP TreeExplainer values on the active model (top seven features), checked so that base value plus contributions equals the predicted probability.	Explainable AI SHAP TreeExplainer Plotly	 *Progress Report Fig 15* *Final Report Sec 5.5, 9.3*

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:  **Date:** 17/10/2026

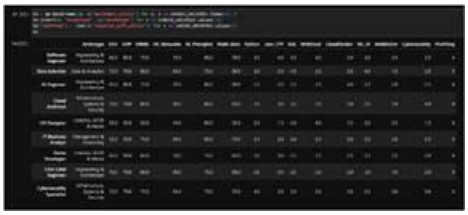
Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	PHGM Dilshani	**Registration No.**	D/BIS/24/0034
Degree / Intake	Information Systems / 2024	**Group No.**	45
Project Title	PathFinder AI: An AI-Based Career Path Prediction and Personalised Career Guidance System		

Technical Contributions:

Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
1. Career knowledge base & skill matrix	Built the knowledge base of nine careers: archetype, benchmark scores for six core modules and eight technical skills, required soft skills from the 24-skill four-pillar framework, preferred work styles, descriptions and role dynamics. Loaded and verified it through the CAREER_UNIVERSE structure used by all engines.	Python pandas JSON knowledge representation	 *Progress Report Fig 3* *Final Report Sec 7.4, Table 4*

2. Data preparation & feature engineering	Developed the StudentProfile class with a unified skill map and the 20-feature normalised vector (modules /100, skills /5, GPA /4, projects /5, soft skills /6, ordinal year and work style). Prepared and checked the synthetic benchmark cohort (450 x 22, no missing values, 50 per career) and label-encoded the nine careers.	Python pandas, NumPy Min-max style normalisation	 *Progress Report Fig 5*  *Progress Report Fig 7*
3. Rule base	Defined the IF-THEN knowledge: GPA bands, internship, project portfolio, career-specific module and skill rules, elective rules, soft-skill ratio bonus and work-style match, with points and explanation text for each rule (later moved to rules.json).	Rule-based reasoning (knowledge-based system) JSON	 *Progress Report Fig 12* *Final Report Sec 5.1*
4. Skill-gap analysis	Implemented gap calculation between the student's profile and the career benchmark for 14 technical competencies plus missing required soft skills, with High / Medium / Low urgency; gaps ranked by size relative to each skill's own scale.	Python pandas Normalised deficit ranking	 *Progress Report Fig 16* *Final Report Sec 5.6, Table 8*

5. Certification catalogue & cosine-similarity recommender	Compiled the 19-certification catalogue (issuer, level, domains, target years, weighted skill vectors) in certifications.json and built the recommender: urgency-weighted gap vector matched by cosine similarity, adjusted for domain relevance and year of study, returning the top three with coverage percentage and links.	Content-based recommendation Cosine similarity (NumPy) JSON	 *Progress Report Fig 4* *Output shown in Progress Report Fig 16 and Fig 25*
6. SQLite database & history log	Created the local SQLite database that stores every assessment (scores, profile, gaps) and supports the history log, summary statistics, CSV export and the comparison of later assessments for progress tracking.	SQLite Python (sqlite3)	 *Progress Report Fig 26* *Final Report Sec 8.7*

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature: Malka **Date:** 17/10/2026

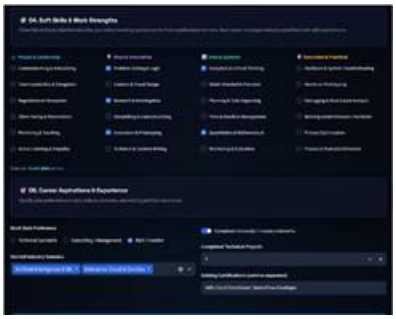
Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.

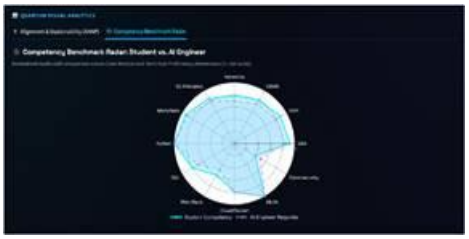
ESSENTIALS OF ARTIFICIAL INTELLIGENCE

INDIVIDUAL CONTRIBUTION SHEET

Student Name	**KHC Vihansa**	**Registration No.**	**D-BCS-25-0022**
Degree / Intake	**Computer Science** **Intake 42**	**Group No.**	**45**
Project Title	**PathFinder AI :** An AI Based Career Path Prediction and Personalized Career Guidance System		

Technical Contributions:

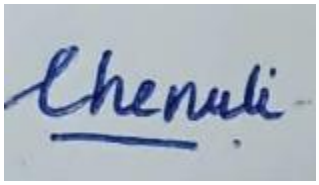
Task / Component	My Contribution	Tools / AI Technique Used	Evidence / Output
1. Streamlit application & profile intake interface	Developed the Streamlit front end (app.py) of PathFinder AI with four tabs: Profile Intake, Guidance & Roadmap, Dataset & Model Studio and History Log. Built the intake form for GPA and academic year, six core module sliders, elective selection with dynamic grade sliders, eight technical proficiency sliders, 24 soft skills in four pillars, work style, desired domains, internship, projects and certifications. Added quick load sample profiles and the "Run Multi-Engine Inference" button.	Python Streamlit widgets and session state Custom CSS dark theme	 *Progress Report Fig 17*  *Progress Report Fig 20*

2. Results dashboard & Plotly visualisations	Created the Guidance & Roadmap page: top 3 career cards with match percentage, confidence band and role dynamic, career alignment chart for all nine careers, SHAP feature contribution chart, soft skill archetype match, collapsible rule firing trace and the competency benchmark radar comparing the student with the career requirement.	Plotly (bar and radar charts) Streamlit HTML / CSS cards	 *Progress Report Fig 21*  *Progress Report Fig 22*  *Progress Report Fig 24*
3. Skill gap and certification display	Designed the Competency & Skill Gap panel with urgency badges and the Recommended Industry Certifications cards, each showing gap coverage, covered skills and links to the official exam syllabus and a preparation course.	Streamlit HTML / CSS cards	 *Progress Report Fig 25*

4. Dataset & Model Studio and History Log interface	Built the interface for choosing the benchmark or an uploaded CSV/Excel dataset, template download, cohort statistics, class balance and GPA charts, model retraining controls (trees, depth, test split) with accuracy and feature importance, and the History Log tab with summary statistics and CSV export.	Streamlit Plotly pandas	 *Progress Report Fig 10*
5. Prepared the Progress Review Report.	Prepared the Progress Review Report.	Technical report writing	Progress Review Report (source of Progress Report Figs 10, 17, 20- 25)

Declaration

I confirm that the contributions listed above accurately represent my individual work in this group project, and I can explain or demonstrate the stated work if required.

Student Signature:		**Date:**	17/10/2026

Evidence may include source code, model/notebook, diagram, dataset work, test results, screenshots, report section or presentation slides.